Decision Trees and Ensemble Methods

EECS 492

Introduction to Artificial Intelligence

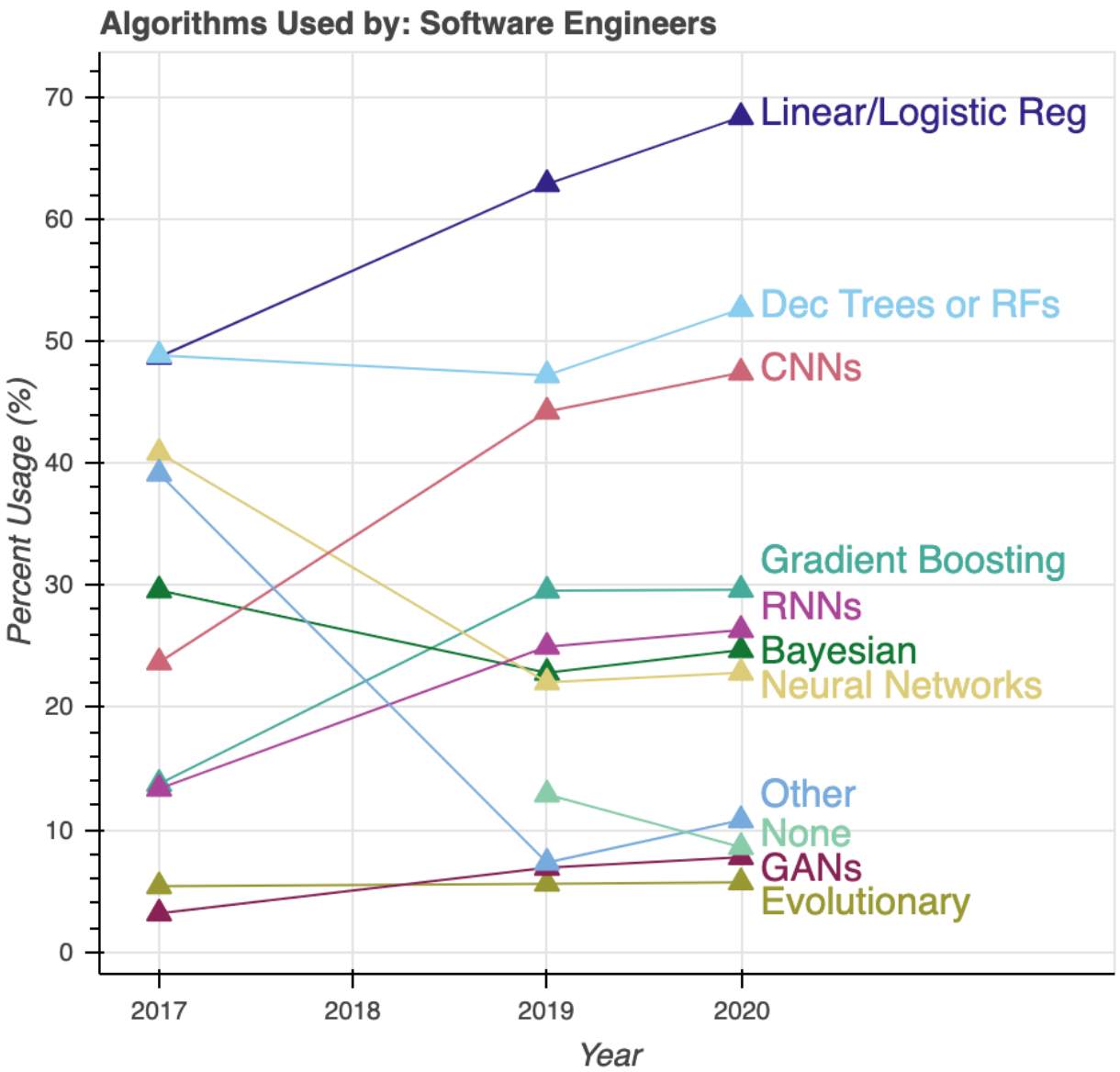
Reminders

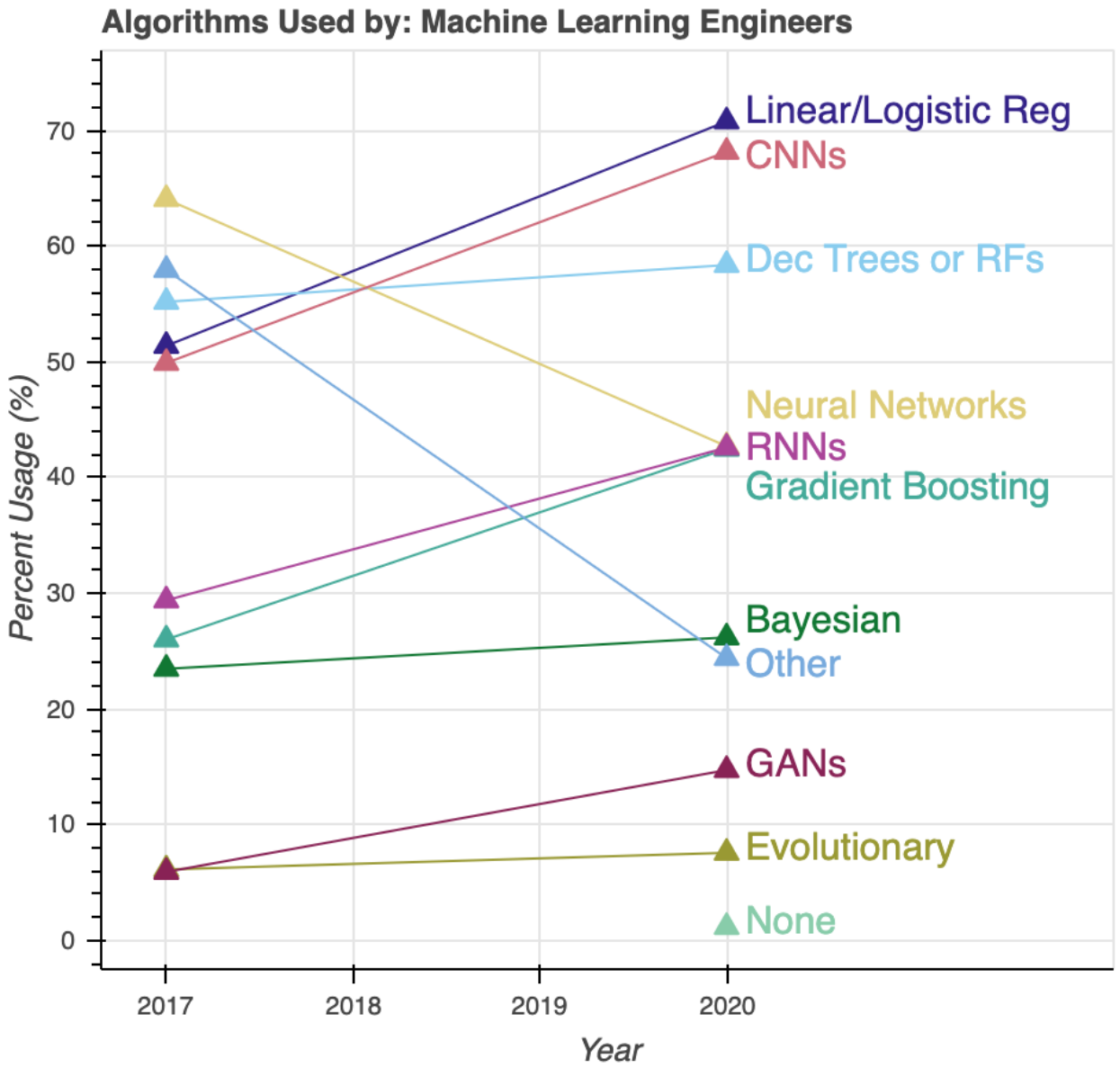
- This week's discussion on decision trees and linear regression

- HW 1 was released last Thursday
  - Start early! due date: Feb 4
  - Topics: Agents and ML
  - Coding resources available in Canvas
    - Files > Coding Resources
    - Includes a video guide on virtual environments

Today's Class

- Decision Trees
- Ensemble Approaches





Source: Kaggle State of Data Science and Machine Learning 2017-2020

An Example Classification Problem

- Target: Will we or won’t we wait for a table at a restaurant

- Attributes:
  - Alternate – do we have another option?
  - Bar – is there a bar area we can wait in?
  - Fri/Sat– is it Friday or Saturday (true/false)?
  - Hungry – are we?
  - Patrons – how many people (none, some, full)?
  - Price – how pricy ($, $$, $$$)?
  - Raining – is it?
  - Reservation – did we make one?
  - Type – food (French, Italian, Thai, burger)?
  - WaitEstimate – how long to wait (0-10, 10-30, 30-60, >60 min)?

The Restaurant Table

Attributes also also known as features (notation: x)

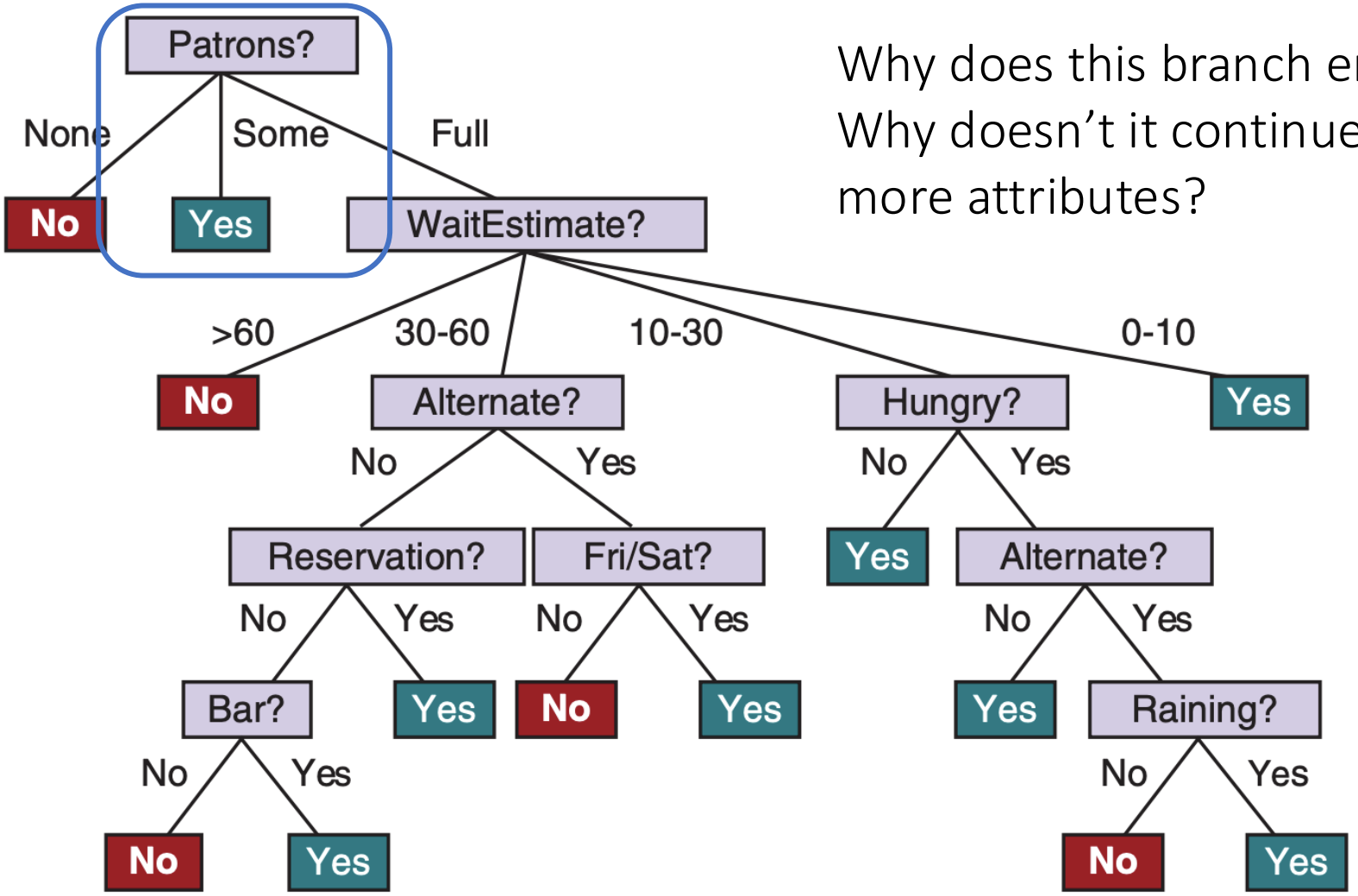
Target or label (notation: y)

Examples, observations, or data points

Example	Input Attributes										Goal
	Alt	*Bar*	*Fri*	*Hun*	*Pat*	*Price*	*Rain*	*Res*	*Type*	*Est*	*WillWait*
$\mathbf{x}_1$	*Yes*	*No*	*No*	*Yes*	*Some*	*$$$*	*No*	*Yes*	*French*	*0–10*	$y_1 = Yes$
$\mathbf{x}_2$	*Yes*	*No*	*No*	*Yes*	*Full*	*$*	*No*	*No*	*Thai*	*30–60*	$y_2 = No$
$\mathbf{x}_3$	*No*	*Yes*	*No*	*No*	*Some*	*$*	*No*	*No*	*Burger*	*0–10*	$y_3 = Yes$
$\mathbf{x}_4$	*Yes*	*No*	*Yes*	*Yes*	*Full*	*$*	*Yes*	*No*	*Thai*	*10–30*	$y_4 = Yes$
$\mathbf{x}_5$	*Yes*	*No*	*Yes*	*No*	*Full*	*$$$*	*No*	*Yes*	*French*	*>60*	$y_5 = No$
$\mathbf{x}_6$	*No*	*Yes*	*No*	*Yes*	*Some*	*$$*	*Yes*	*Yes*	*Italian*	*0–10*	$y_6 = Yes$
$\mathbf{x}_7$	*No*	*Yes*	*No*	*No*	*None*	*$*	*Yes*	*No*	*Burger*	*0–10*	$y_7 = No$
$\mathbf{x}_8$	*No*	*No*	*No*	*Yes*	*Some*	*$$*	*Yes*	*Yes*	*Thai*	*0–10*	$y_8 = Yes$
$\mathbf{x}_9$	*No*	*Yes*	*Yes*	*No*	*Full*	*$*	*Yes*	*No*	*Burger*	*>60*	$y_9 = No$
$\mathbf{x}_{10}$	*Yes*	*Yes*	*Yes*	*Yes*	*Full*	*$$$*	*No*	*Yes*	*Italian*	*10–30*	$y_{10} = No$
$\mathbf{x}_{11}$	*No*	*No*	*No*	*No*	*None*	*$*	*No*	*No*	*Thai*	*0–10*	$y_{11} = No$
$\mathbf{x}_{12}$	*Yes*	*Yes*	*Yes*	*Yes*	*Full*	*$*	*No*	*No*	*Burger*	*30–60*	$y_{12} = Yes$

When poll is active respond at PollEv.com/492

A Decision Tree for the Restaurant Example



Why does this branch end at "Yes"?
Why doesn't it continue to branch on more attributes?

Decision Tree Classification

- Simple supervised learning model for classification

- Structure:
  - Input: vector of attribute values (A_i)
  - Output: decision

- How is decision made?
  - Sequence of tests
  - Internal nodes – test of attribute values
  - Branch – labeled with possible values of attribute
  - Leaf – value to be returned by function

Simplification: Boolean Classification!

Inducing DTs from Examples

- Let's consider the restaurant example again
  - n = 10 variables
  - How do we find the smallest tree possible (that is consistent with the data)?

- Intractable problem: 2^{2^n} trees!
  - What if we employ some heuristics?
  - We can find an approximate solution!

- This gives us motivation for the DECISION-TREE-LEARNING algorithm!

Overview of DECISION-TREE-LEARNING

- Greedy divide-and-conquer algorithm
  - Always test the most important attribute first!
  - Creates sets of smaller problems – solve recursively
- What does "most important" mean?
  - General: the one that has the biggest impact on the classification of an example
- Goal – correct classification with smallest number of tests
  - Short paths
  - Shallow trees!

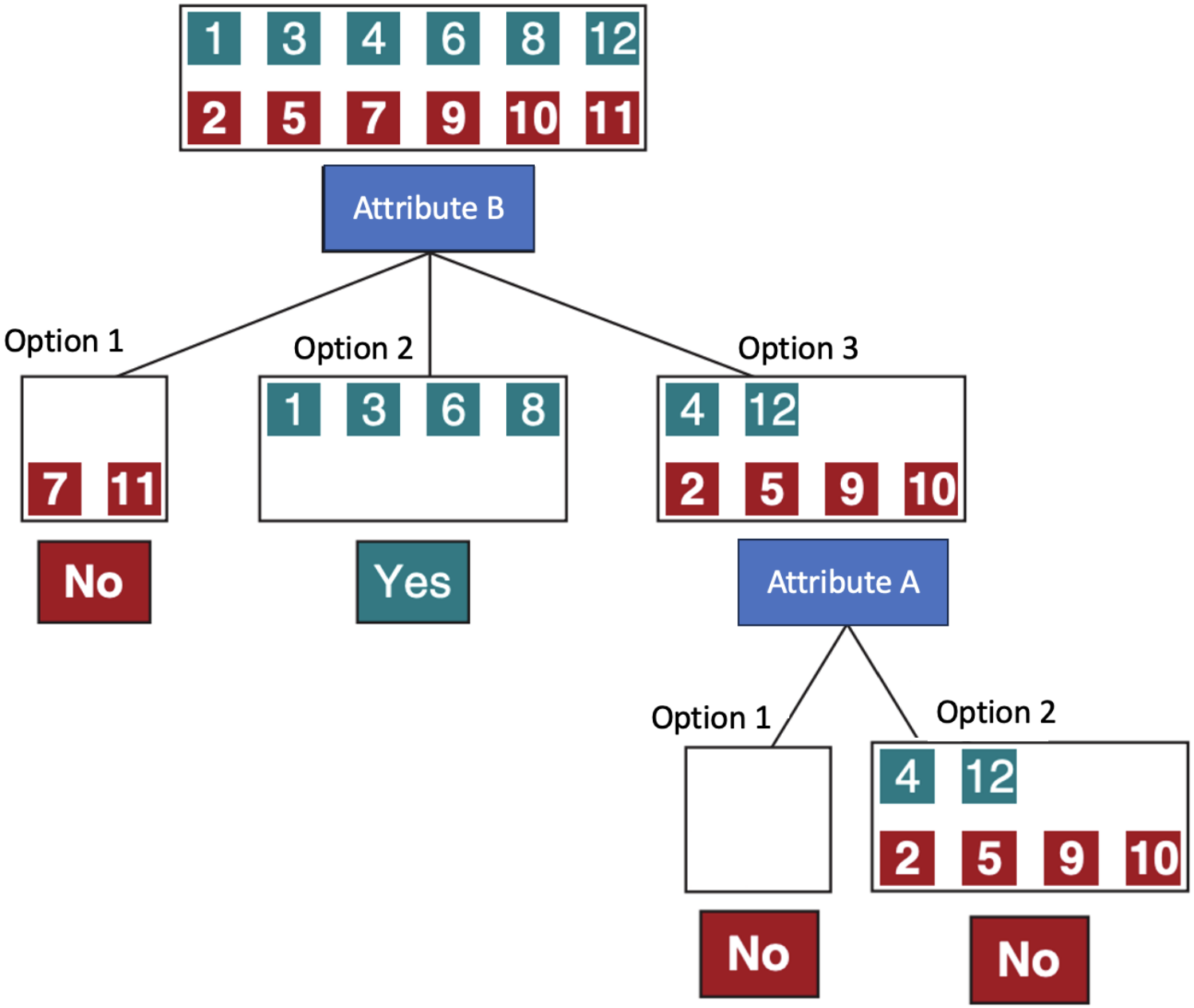
DECISION-TREE-LEARNING

```
function LEARN-DECISION-TREE (examples, attributes, parent_examples) returns a tree

    if examples is empty then return PLURALITY-VALUE(parent_examples)
    else if all examples have the same classification then return classification
    else if attributes is empty then return PLURALITY-VALUE(examples)
    else
        A ← argmax_{a ∈ attributes} IMPORTANCE(a, examples)
        tree ← a new decision tree with root test A
        for each value v of A do
            exs ← {e: e ∈ examples and e.A = v}
            subtree ← LEARN-DECISION-TREE(exs, attributes–A, examples)
            add a branch to tree with label (A = v) and subtree subtree
        return tree
```

Example DT

- Positive examples – blue
- Negative examples - red



Sketch of DECISION-TREE-LEARNING

- First attribute test splits up examples
  - Each outcome is a new DT learning problem!

- Four cases:
  1. Are all remaining cases positive (or negative)? – Done!
  2. Are there some positive and some negative? – Find the best attribute and split!
  3. No examples left? No data supports combination of attribute values – return default value from plurality
  4. No attributes left but still positive and negative examples? – Error or noise in the data – return plurality

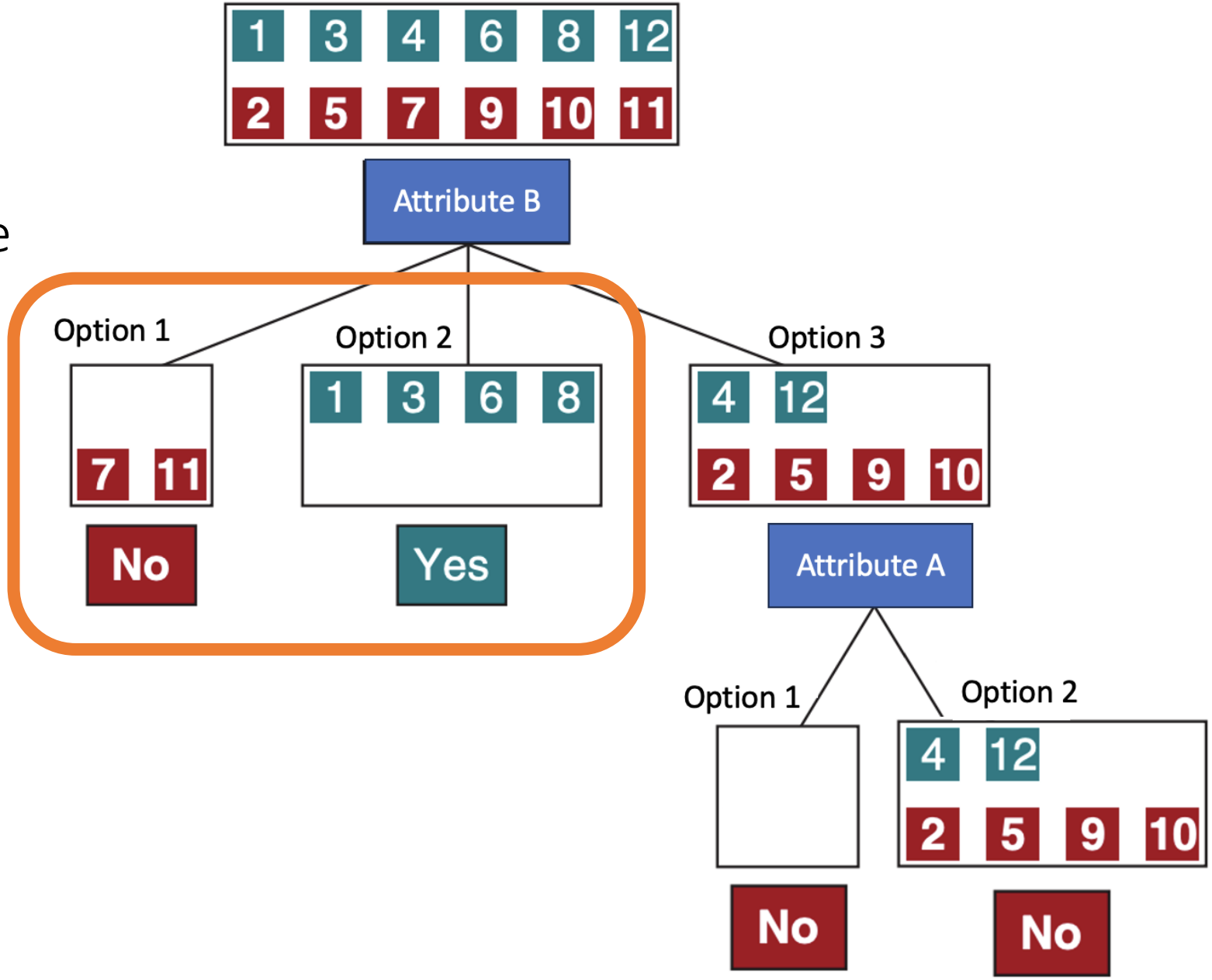
Sketch of DECISION-TREE-LEARNING

- First attribute test splits up examples
  - Each outcome is a new DT learning problem!

- Four cases:
  1. Are all remaining cases positive (or negative)? – Done!
  2. Are there some positive and some negative? – Find the best attribute and split!
  3. No examples left? No data supports combination of attribute values – return default value from plurality of parent
  4. No attributes left but still positive and negative examples? – Error or noise in the data – return plurality

Example DT

- Positive examples – blue
- Negative examples - red



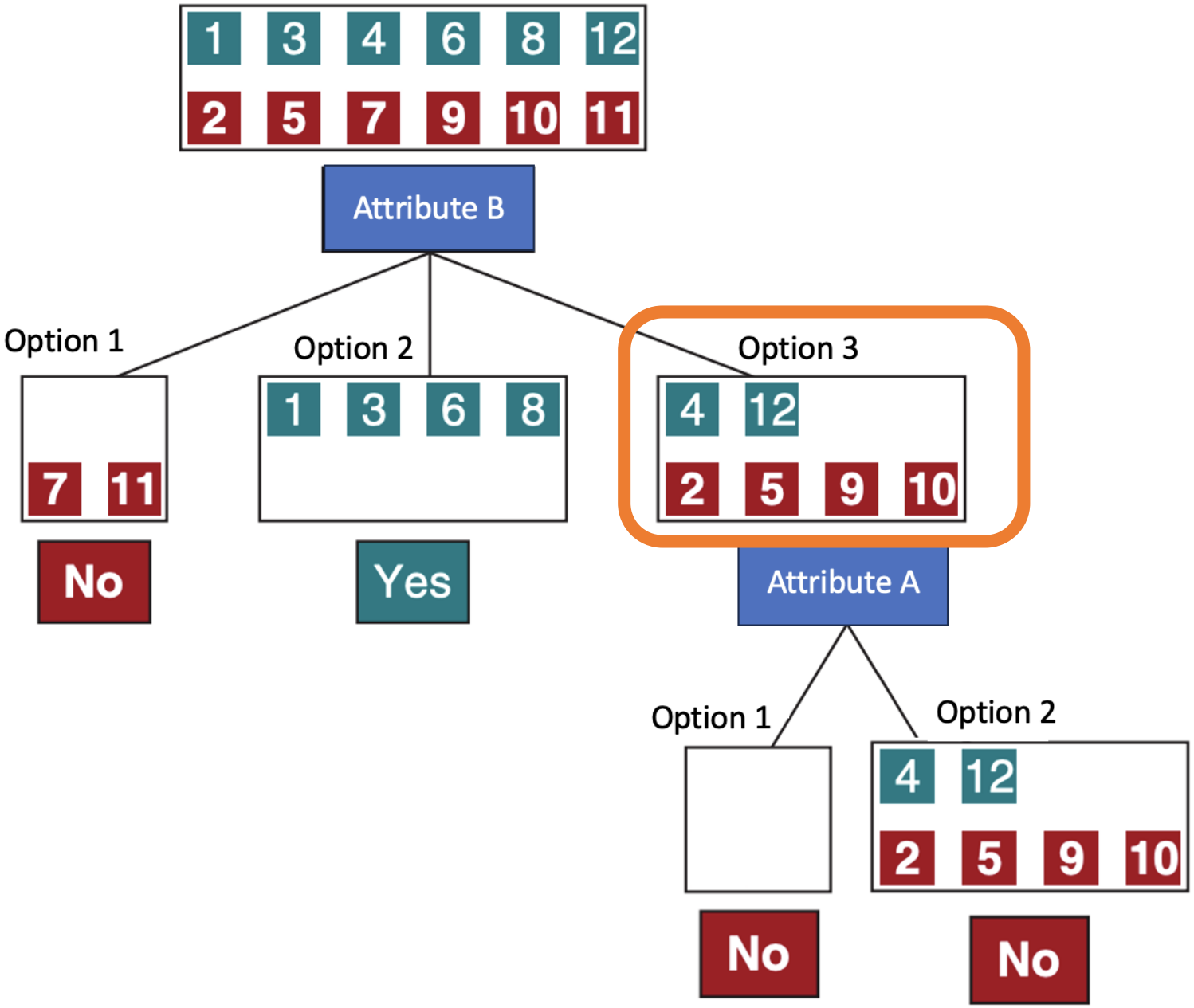
Sketch of DECISION-TREE-LEARNING

- First attribute test splits up examples
  - Each outcome is a new DT learning problem!

- Four cases:
  1. Are all remaining cases positive (or negative)? – Done!
  2. Are there some positive and some negative? – Find the best attribute and split!
  3. No examples left? No data supports combination of attribute values – return default value from plurality of parent
  4. No attributes left but still positive and negative examples? – Error or noise in the data – return plurality

Example DT

- Positive examples – blue
- Negative examples - red



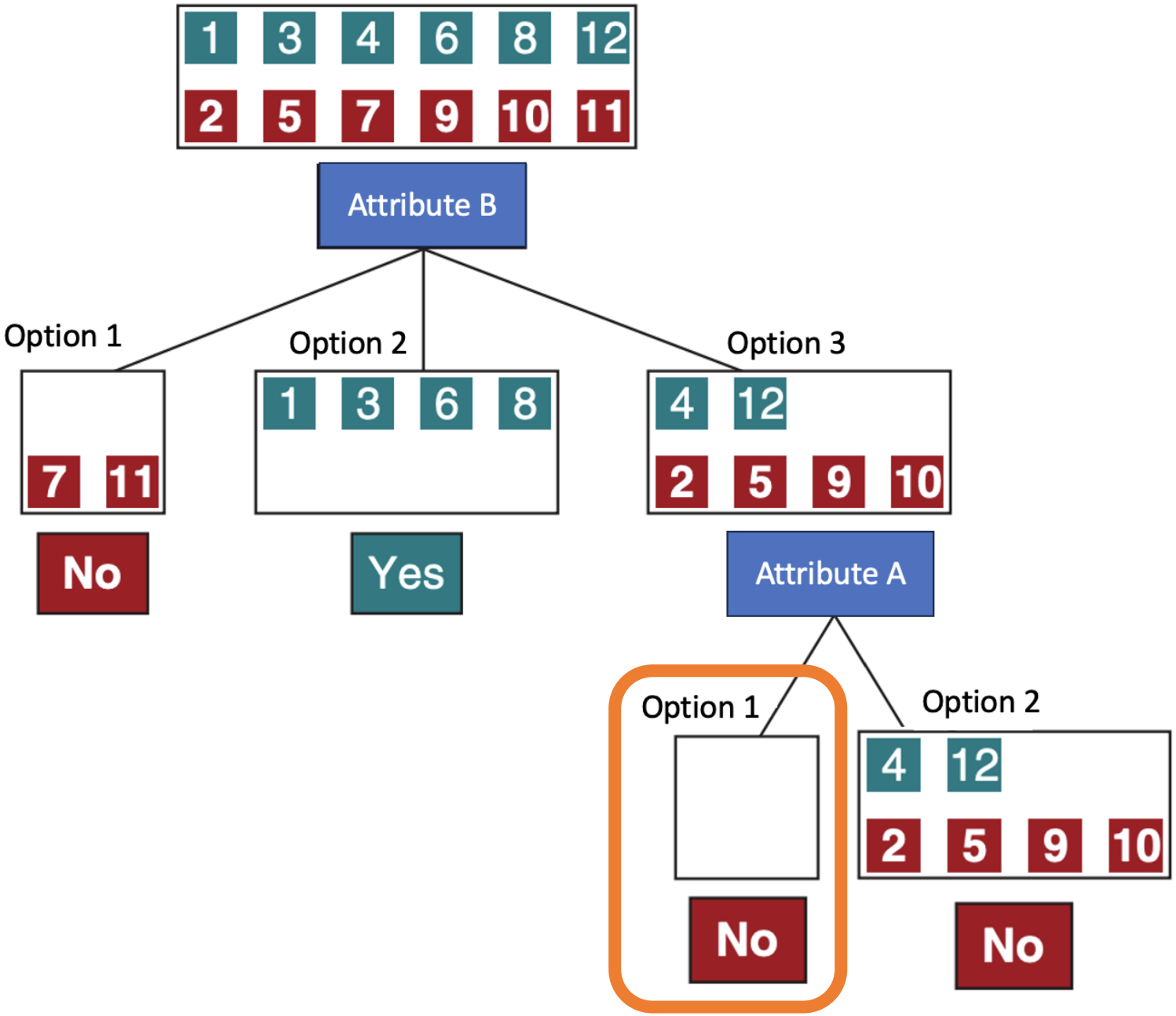
Sketch of DECISION-TREE-LEARNING

- First attribute test splits up examples
  - Each outcome is a new DT learning problem!

- Four cases:
  1. Are all remaining cases positive (or negative)? – Done!
  2. Are there some positive and some negative? – Find the best attribute and split!
  3. No examples left? No data supports combination of attribute values – return default value from plurality of parent
  4. No attributes left but still positive and negative examples? – Error or noise in the data – return plurality

Example DT

- Positive examples – blue
- Negative examples - red



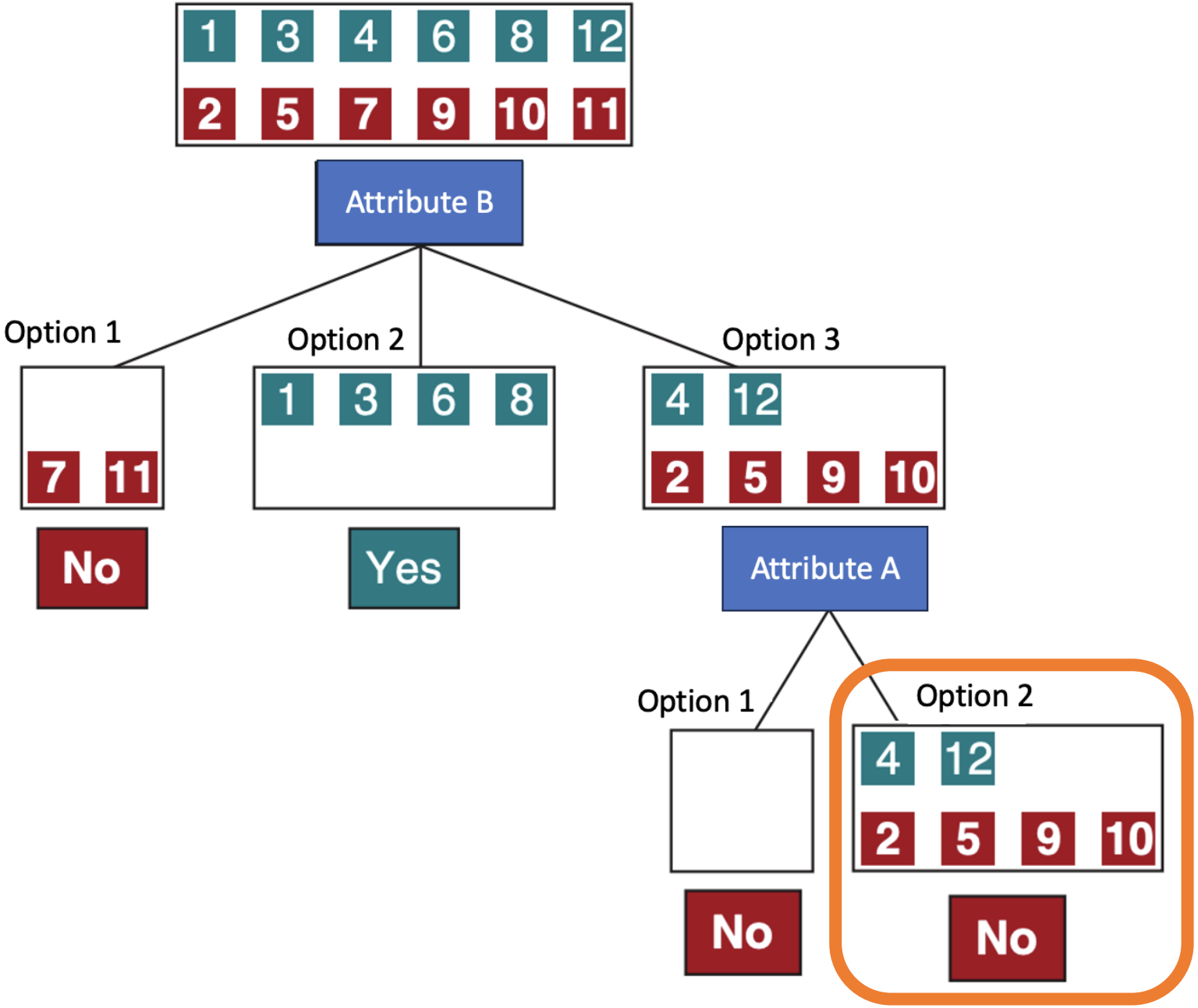
Sketch of DECISION-TREE-LEARNING

- First attribute test splits up examples
  - Each outcome is a new DT learning problem!

- Four cases:
  1. Are all remaining cases positive (or negative)? – Done!
  2. Are there some positive and some negative? – Find the best attribute and split!
  3. No examples left? No data supports combination of attribute values – return default value from plurality of parent
  4. No attributes left but still positive and negative examples? – Error or noise in the data – return plurality

Example DT

- Positive examples – blue
- Negative examples - red



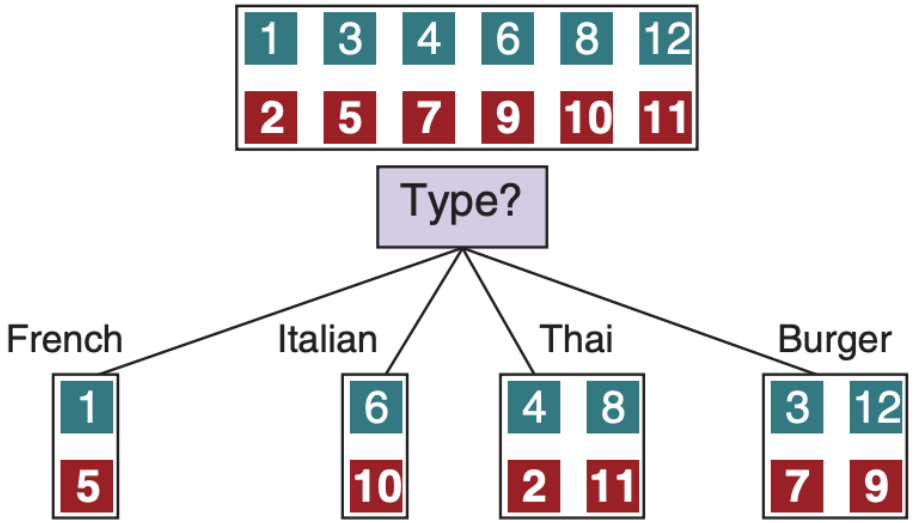
DT Points to Consider

- Important notes:
  - Examples are crucial to tree construction
  - Examples do not appear in the tree itself
  - Tree: just tests on attribute values

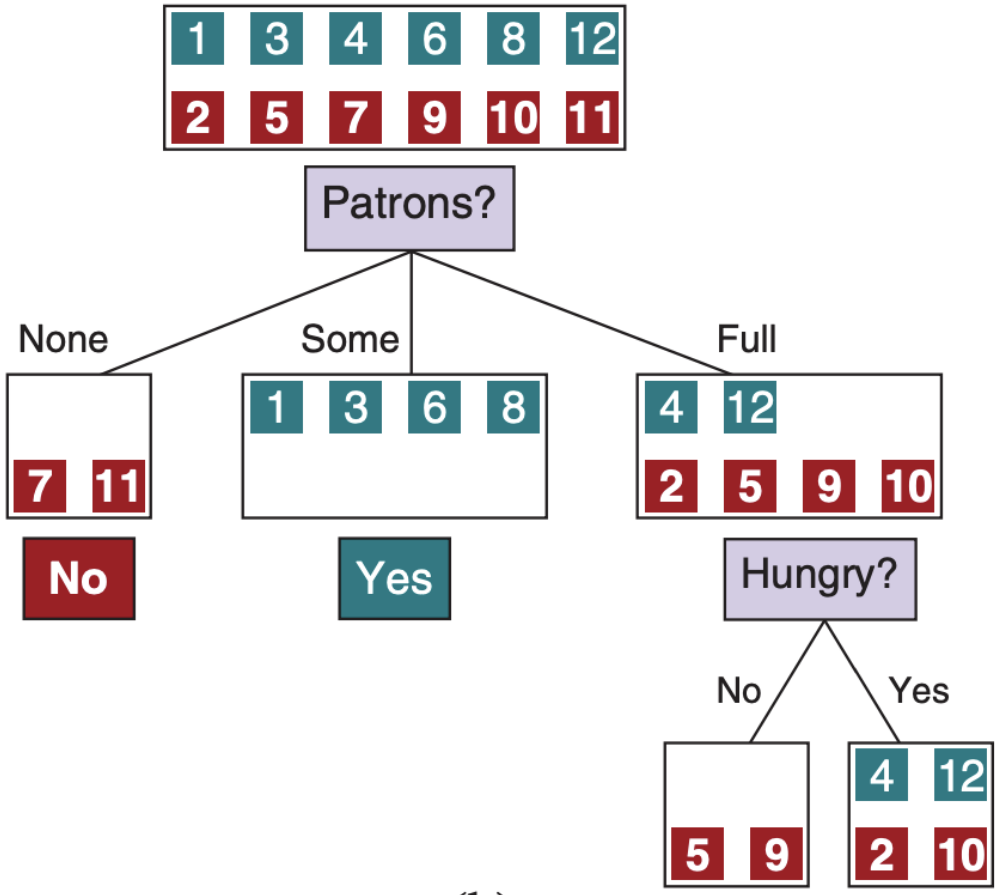
- When might the DT make mistakes
  - Errors in the data
  - Noise in the data
  - Cases where it has seen no examples

When poll is active respond at PollEv.com/492

Which tree is better?



(a)



(b)

DECISION-TREE-LEARNING

function LEARN-DECISION-TREE (*examples, attributes, parent_examples)* **returns** a tree

```
if examples is empty then return PLURALITY-VALUE(parent_examples)
else if all examples have the same classification then return classification
else if attributes is empty then return PLURALITY-VALUE(examples)
else
    A ← argmax_{a ∈ attributes} IMPORTANCE(a, examples)
    tree ← a new decision tree with root test A
    for each value v of A do
        exs ← {e: e ∈ examples and e.A = v}
        subtree ← LEARN-DECISION-TREE(exs, attributes–A, examples)
        add a branch to tree with label (A = v) and subtree subtree
    return tree
```

How to Choose Attribute Tests

- Our algorithm hinges on finding the *most important* attributes to split on – how can we measure this?

- We will use a measure called information gain
  - Split on the attribute with the highest information gain

How can we calculate information gain?

- Information gain is defined in terms of entropy

- Entropy (H) = a measure of the uncertainty of a random variable
  - More information, less entropy

- Examples
  - A coin that always comes up heads $\rightarrow$ 0 bits
  - A fair coin (equally likely to be heads or tails) $\rightarrow$ 1 bits
  - Fair four-sided die $\rightarrow$ 2 bits

Entropy

- We define entropy as:

$$H(V) = \sum_k P(v_k) log_2 \frac{1}{P(v_k)} = -\sum_k P(v_k) log_2 P(v_k)$$

- V is a random variable with values v_k having probability $P(v_k)$

- Example: entropy of a fair coin

When poll is active respond at PollEv.com/492

Exercise: Entropy

- Calculate the entropy of a loaded coin where $P(heads) = 0.99$

- Hw: Calculate the entropy of a four-sided die

Entropy for Boolean Random Variables

- Let's define $B(q)$: the entropy of a Boolean random variable that is true with probability q

$$B(q) = -(q log_2 q + (1-q) log_2 (1-q))$$

- The loaded coin (from the previous slide) is a Boolean random variable, $q = 0.99$
  - $H(Loaded) = B(q) \approx 0.08$

Back to decision trees…

- We are trying to find a way to choose which attribute to split on
  - Split on the attribute with the highest information gain

- Information gain of attribute A = the expected reduction of entropy after splitting on A

$$Gain(A) = H(Goal) - Remainder(A)$$

The entropy of the output variable (before splitting on A)

The expected entropy remaining after splitting on A

Calculating Gain

$$Gain(A) = H(Goal) - Remainder(A)$$

The entropy of the output variable (before splitting on A)

The expected entropy remaining after splitting on A

- How can we calculate $H(Goal)$?
  - This is the entropy of the goal attribute
  - Let's say we have p positive examples and n negative examples

$$H(Goal) = B\left(\frac{p}{p+n}\right)$$

$$H(Goal) = -\left(\frac{p}{p+n} log_2 \frac{p}{p+n} + \left(1 - \frac{p}{p+n}\right) log_2 \left(1 - \frac{p}{p+n}\right)\right)$$

Probability of the positive class

Probability of the negative class

Exercise: What is $H(Goal)$?

Example	Input Attributes										Goal
	Alt	*Bar*	*Fri*	*Hun*	*Pat*	*Price*	*Rain*	*Res*	*Type*	*Est*	*WillWait*
$\mathbf{x}_1$	*Yes*	*No*	*No*	*Yes*	*Some*	*\$\$\$*	*No*	*Yes*	*French*	*0–10*	$y_1 = Yes$
$\mathbf{x}_2$	*Yes*	*No*	*No*	*Yes*	*Full*	*\$*	*No*	*No*	*Thai*	*30–60*	$y_2 = No$
$\mathbf{x}_3$	*No*	*Yes*	*No*	*No*	*Some*	*\$*	*No*	*No*	*Burger*	*0–10*	$y_3 = Yes$
$\mathbf{x}_4$	*Yes*	*No*	*Yes*	*Yes*	*Full*	*\$*	*Yes*	*No*	*Thai*	*10–30*	$y_4 = Yes$
$\mathbf{x}_5$	*Yes*	*No*	*Yes*	*No*	*Full*	*\$\$\$*	*No*	*Yes*	*French*	*>60*	$y_5 = No$
$\mathbf{x}_6$	*No*	*Yes*	*No*	*Yes*	*Some*	*\$\$*	*Yes*	*Yes*	*Italian*	*0–10*	$y_6 = Yes$
$\mathbf{x}_7$	*No*	*Yes*	*No*	*No*	*None*	*\$*	*Yes*	*No*	*Burger*	*0–10*	$y_7 = No$
$\mathbf{x}_8$	*No*	*No*	*No*	*Yes*	*Some*	*\$\$*	*Yes*	*Yes*	*Thai*	*0–10*	$y_8 = Yes$
$\mathbf{x}_9$	*No*	*Yes*	*Yes*	*No*	*Full*	*\$*	*Yes*	*No*	*Burger*	*>60*	$y_9 = No$
$\mathbf{x}_{10}$	*Yes*	*Yes*	*Yes*	*Yes*	*Full*	*\$\$\$*	*No*	*Yes*	*Italian*	*10–30*	$y_{10} = No$
$\mathbf{x}_{11}$	*No*	*No*	*No*	*No*	*None*	*\$*	*No*	*No*	*Thai*	*0–10*	$y_{11} = No$
$\mathbf{x}_{12}$	*Yes*	*Yes*	*Yes*	*Yes*	*Full*	*\$*	*No*	*No*	*Burger*	*30–60*	$y_{12} = Yes$

Calculating Gain

$$Gain(A) = H(Goal) - Remainder(A)$$

The entropy of the output variable (before splitting on A)

The expected entropy remaining after splitting on A

- How can we calculate $Remainder(A)$?
  - Let's say attribute A has d distinct values
  - Divides training set into subsets $E_1, \ldots, E_d$
  - Each subset E_k has p_k positive, n_k negative examples
  - If we split the decision tree on A, and go along the branch with subset E_k, how many bits of information will we need to answer the question?

$$B\left(\frac{p_k}{p_k+n_k}\right)$$

  - Summing over all values k, we can take the *expected entropy:*

$$Remainder(A) = \sum_{k=1}^{d} \frac{p_k + n_k}{p + n} B\left(\frac{p_k}{p_k + n_k}\right)$$

How much randomness is in the examples with that value?

How likely is the kth value?

Exercise: Gain

$$Gain(A) = H(Goal) - Remainder(A)$$

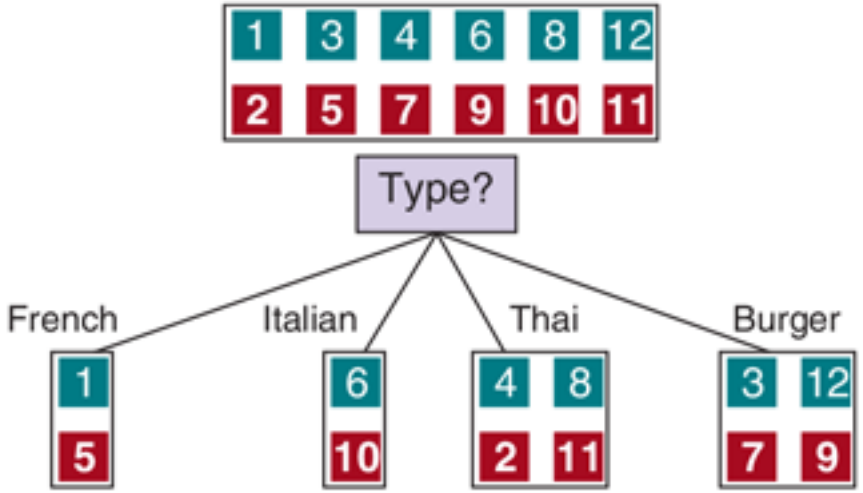
The entropy of the output variable (before splitting on A)

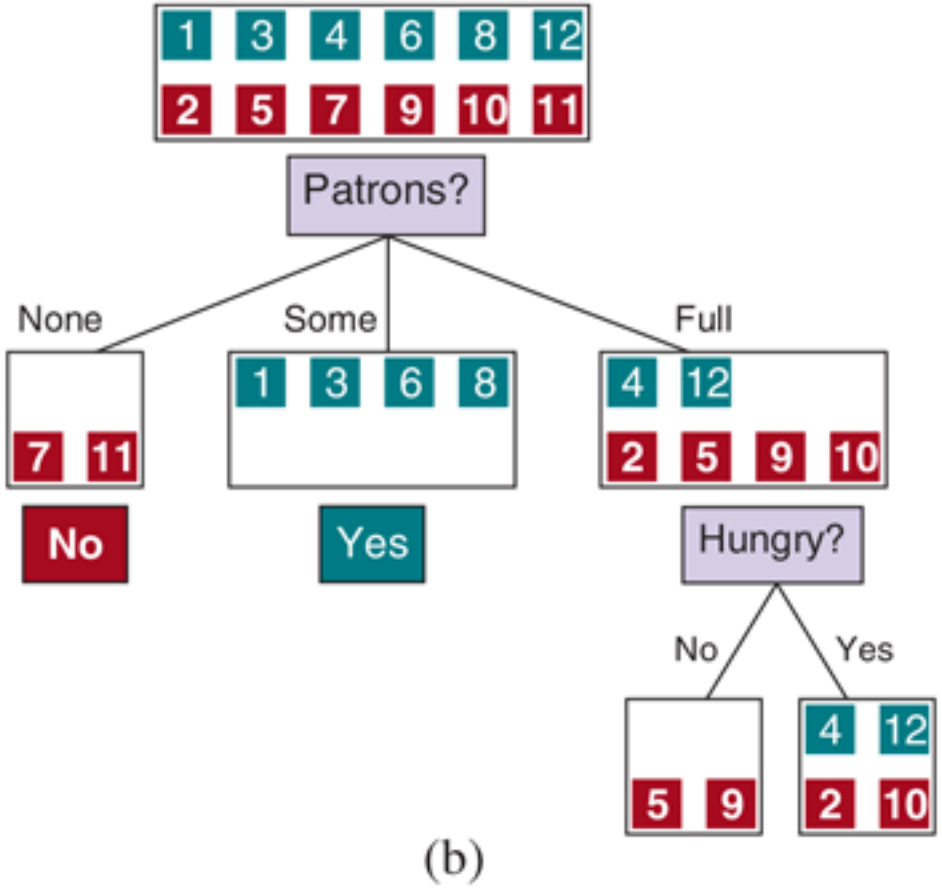
The expected entropy remaining after splitting on A

- What is the information gain of Patrons?
- What is the information gain of Type?
- If you were just choosing between these two attributes, which attribute would you split on?

When poll is active respond at PollEv.com/492

Let's Analyze Two of the Possible Splits





(b)

$$B(q) = -\left(q \log_2 q + (1-q)\log_2(1-q)\right)$$

$$Remainder(A) = \sum_{k=1}^{d} \frac{p_k+n_k}{p+n} B\left(\frac{p_k}{p_k+n_k}\right)$$

$$Gain(A) = B\left(\frac{p}{p+n}\right) - Remainder(A)$$

$$B(1) = 0; B(0) = 0; B(1/2) = 1$$

$$B(1/3) = B(2/3) \approx 0.92; B(1/4) = B(3/4) \approx 0.81$$

Problems with DT

- Are trees always appropriate?
  - Think about rolling a die – does a tree make sense?
- What is the danger?
  - Overfitting – creating a model that closely matches your training data without knowing if it can generalize!
- Possible solution
  - Decision tree pruning
  - Remove tests that have only noise as their outcome [for more detail, check out chapter 19.3.4]
  - Have many small trees and combine them → Ensemble methods!

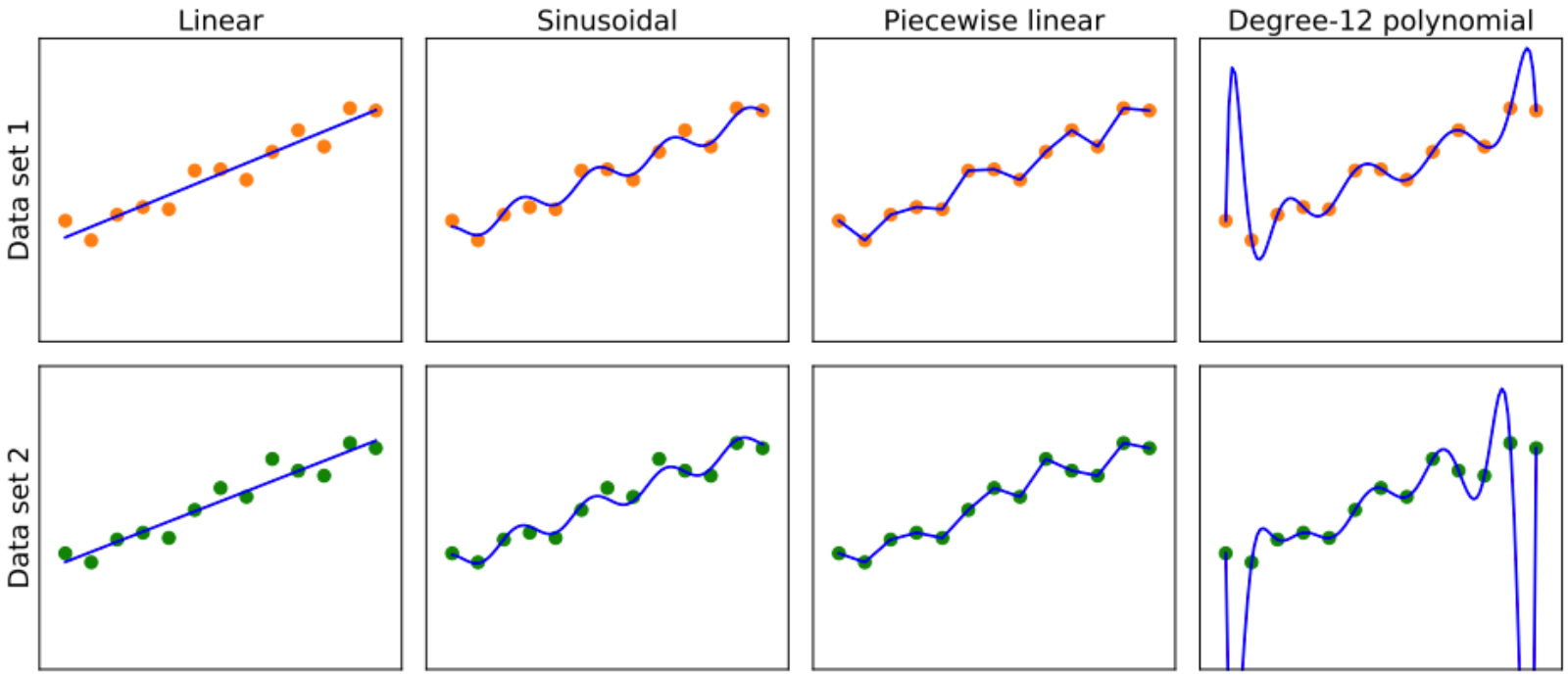
Ensemble Approaches

Overview

- Create a collection (ensemble) of hypotheses (base models)
- Combine their predictions by averaging, voting, or another level of machine learning
- Combined model: ensemble model

Remember Bias-Variance Trade-off?

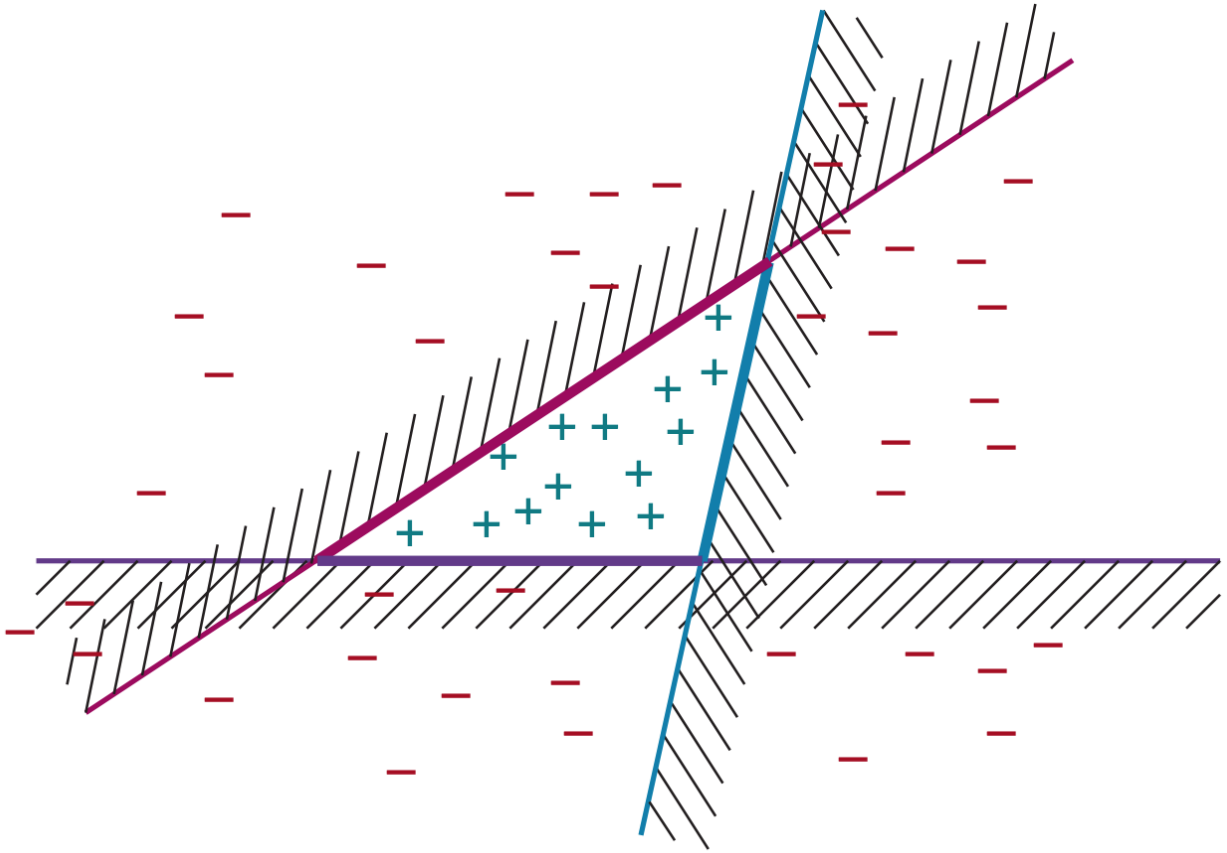
- Review: Which models have low bias? High bias? Low variance? High variance?



Why use an ensemble approach?

- Benefit 1: Reduce bias (base models may be too restrictive)

- Example: three linear classifiers can represent a triangular region (one linear classifier can't do this!)



Why use an ensemble approach?

- Benefit 2: Reduce variance

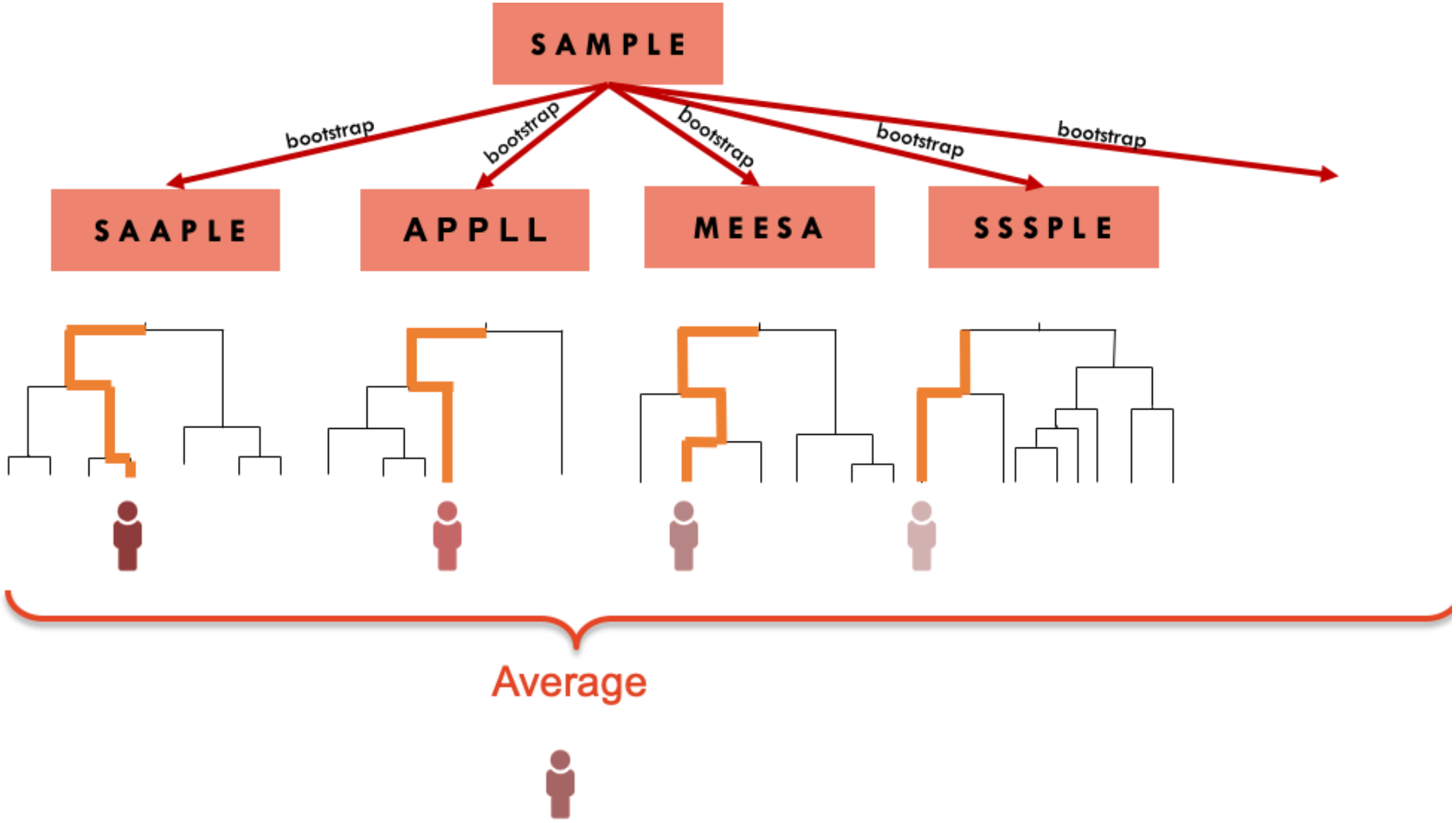
- Let's say we have 5 binary classifiers, combined using majority voting
  - In order to be incorrect, 3 out of 5 classifiers must be wrong
  - If each classifier is correct 75% of the time, the ensemble will be correct 89% of the time!
  - If you have 17 classifiers correct 75% of the time, the ensemble will be correct 99% of the time!

Bagging

- Steps:
  - Generate *K* distinct training sets by sampling with replacement from the original training set
  - Train classifiers on each distinct training set

- Make a decision:
  - Classification: majority vote
  - Regression: take the average - $h(x) = \frac{1}{K}\sum_{i=1}^{K} h_i(x)$

- Benefit: tends to reduce variance, used commonly in decision trees



https://bookdown.org/tpinto_home/Beyond-Additivity/bagging.html

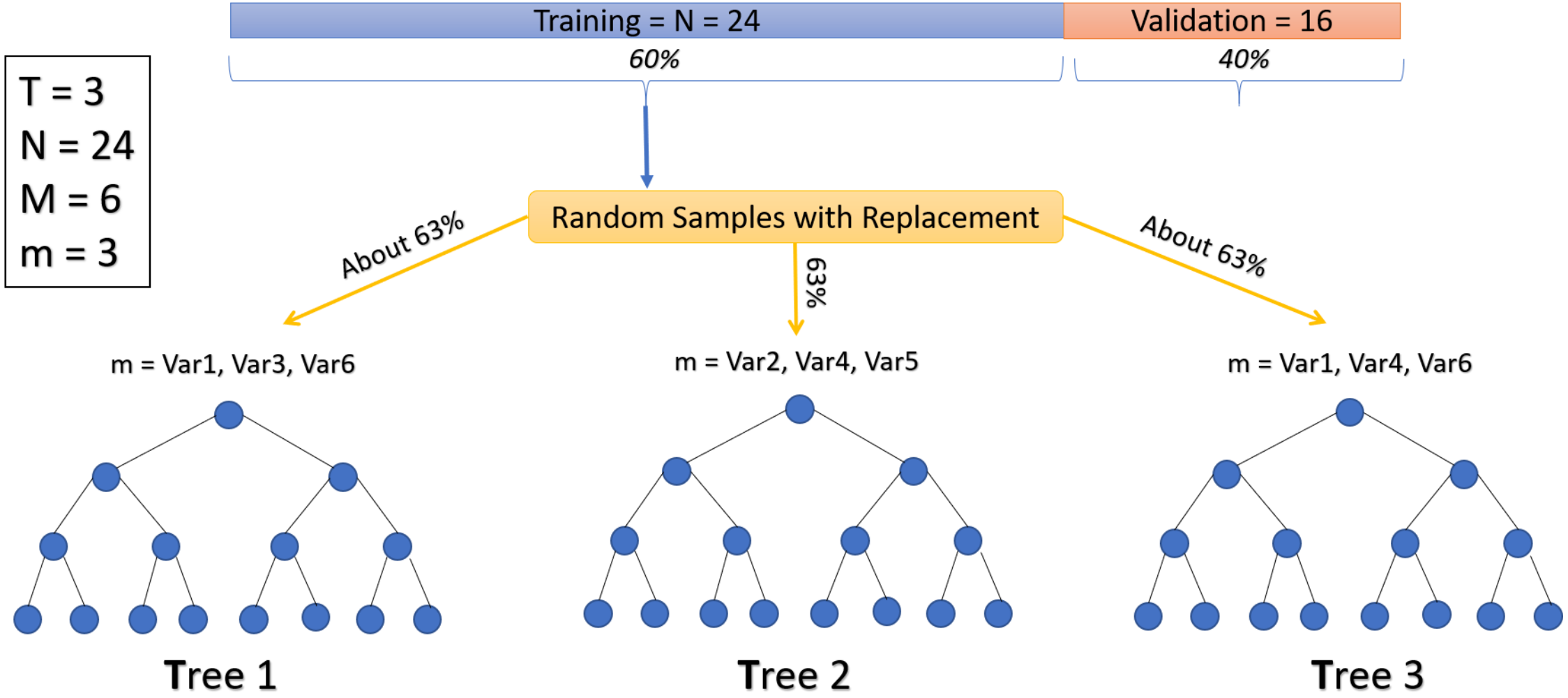
Random Forests (RF)

- Motivation:
  - Bagging decision trees produces very similar trees!
  - Take steps to introduce diverse trees

- Random forest:
  - Key idea: randomly vary *attribute choices* rather than training examples
  - At each split point, select a random sampling of attributes, use the attribute with the highest information gain
  - Add bagging as well

- Benefit: Complex, unpruned models

Random Forest Simplified



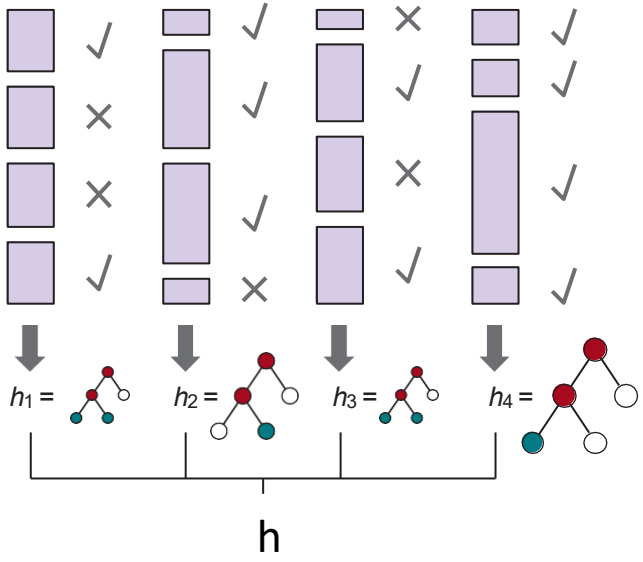
https://app.myeducator.com/reader/web/1421a/15/xu4rw/

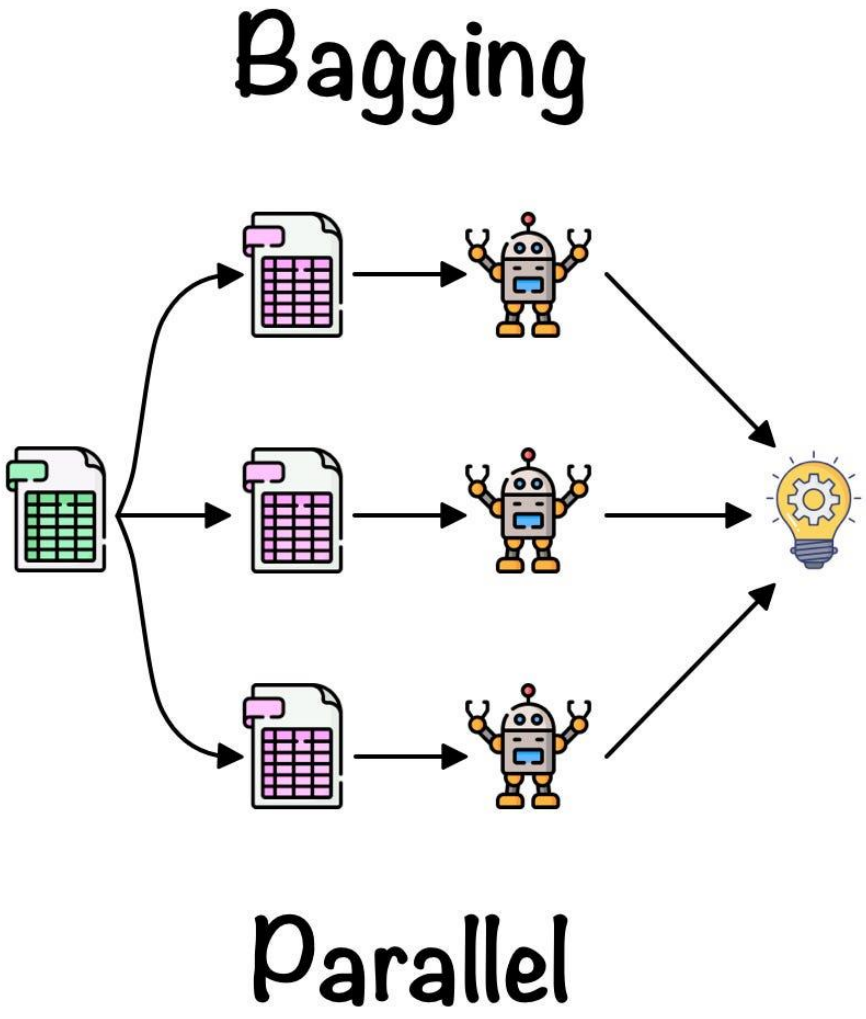
Boosting

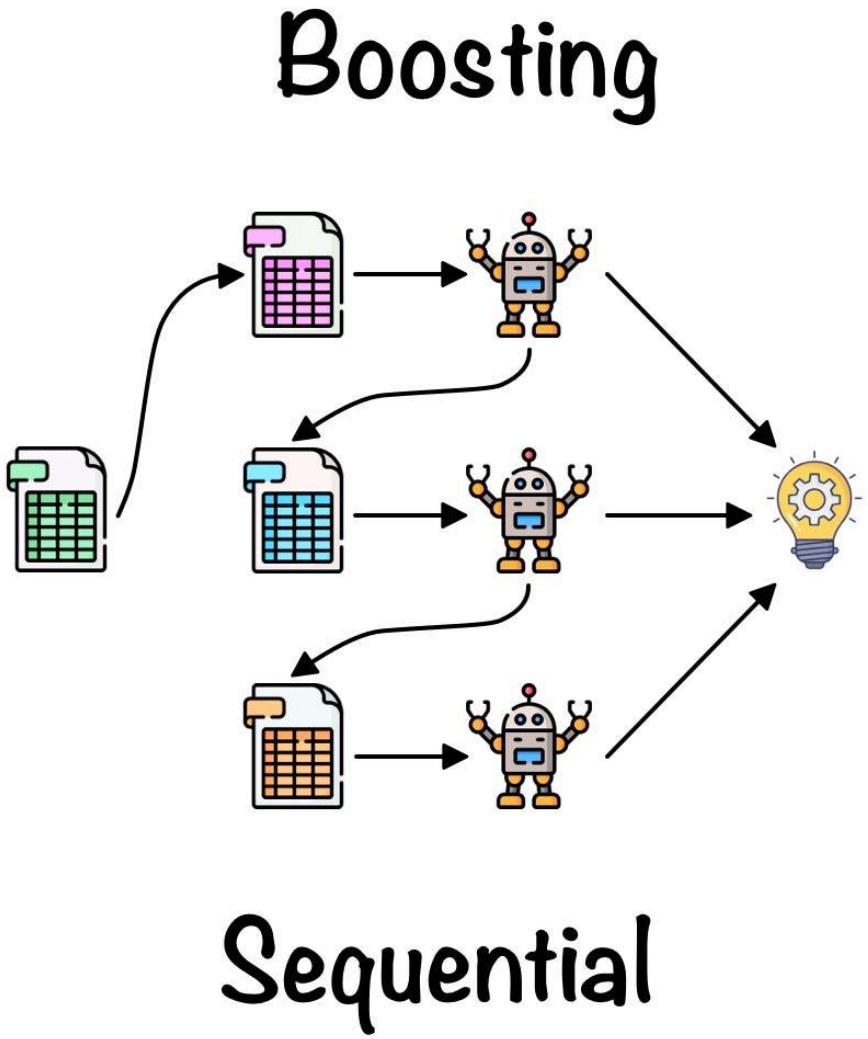
- Key component: a weighted training set (each sample has a weight, $w_j \geq 0$) – think of it as having multiple copies of the same example

- Steps:
  1. Start with equal weights, generate first hypothesis (h_1)
  2. Increase weights that h_1 misclassified, decrease weights of correctly classified examples
  3. Then, generate next hypothesis (h_2)
  4. Continue until *K* hypotheses have been generated
  5. Then, vote over all hypotheses: $h(\mathbf{x}) = \sum_{i=1}^{K} z_i h_i(\mathbf{x})$ (z_i are new weights)
     1. Hypotheses that did better on their respective weighted training sets have more weight

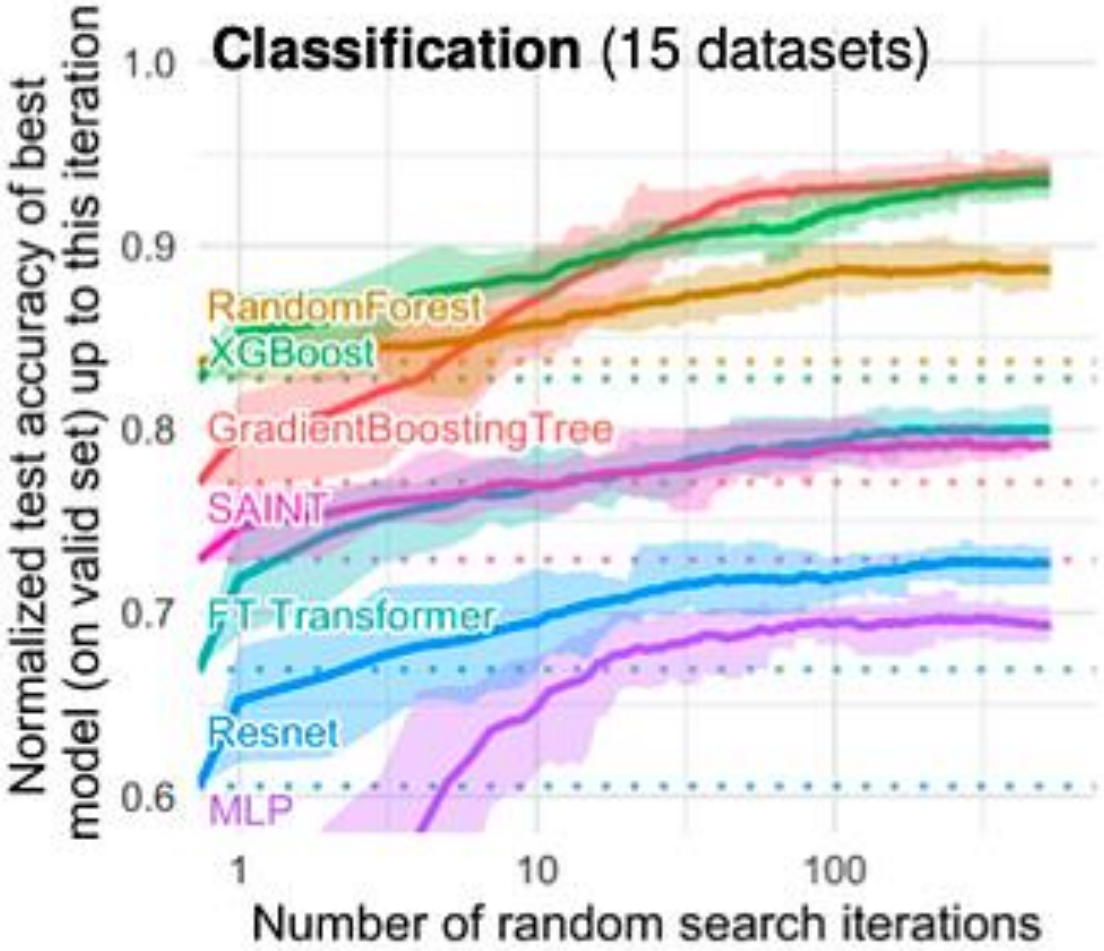
Boosting, graphically

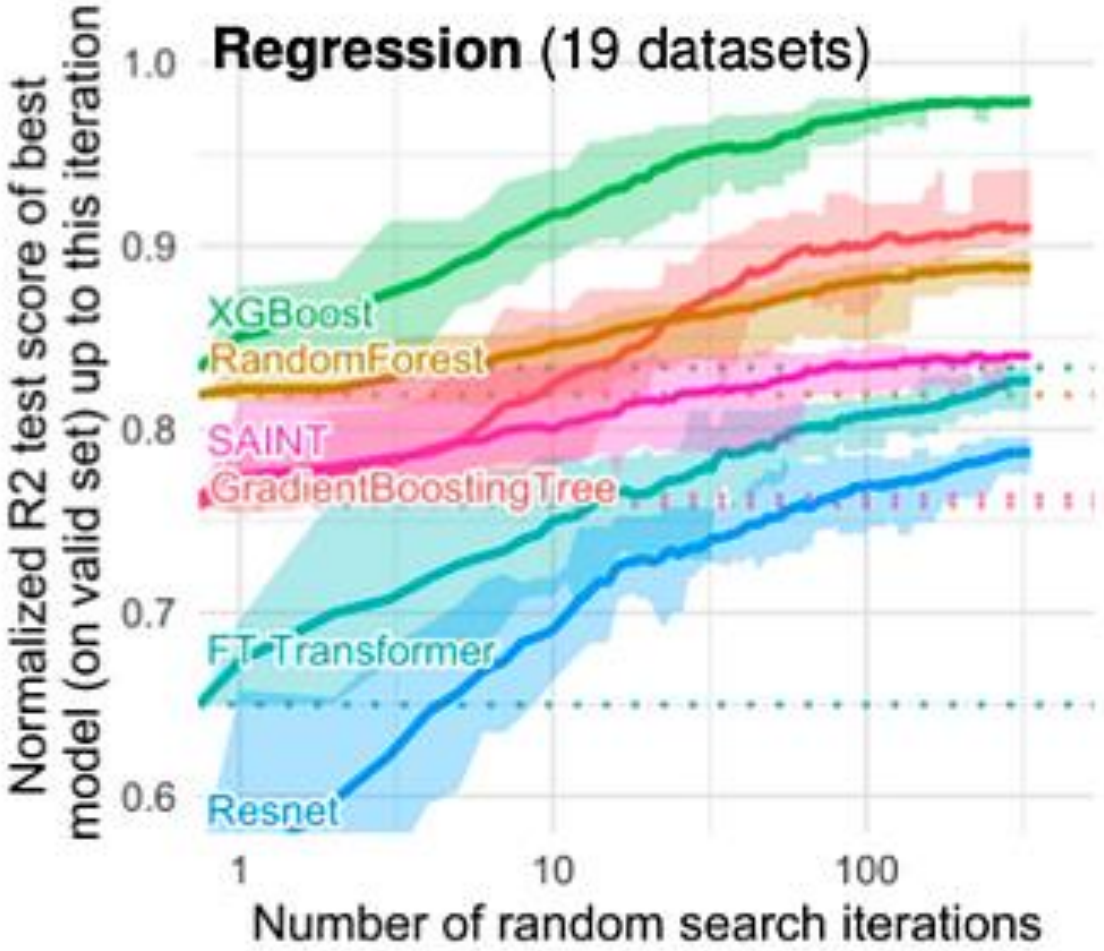






https://towardsdatascience.com/ensemble-learning-bagging-boosting-3098079e5422





Grinsztajn and Oyallon. Why do tree-based models still outperform deep learning on tabular data? NeurIPS 2022

When poll is active respond at **PollEv.com/492**

Why decision trees and variants are some of the most preferred methods in machine learning? Write three keywords.

I thought deep learning was all you need…